Regular Expression

"Filtering data through RegEx"

Table of Contents

01 Kursplan & Översikt

02 What is it?

03 Regular Expression

04 Eget Arbete & Uppgifter

Overview

Moduler:

- Regex theory
- Regex Code Examples
- Filtering & Conditions

02

What is it?

Regular Expression
(What is it?)

Regular expressions (called REs, or regexes, or regex patterns) are essentially a tiny, highly specialized programming language embedded inside Python and made available through the `re` module. Using this little language, you specify the rules for the set of possible strings that you want to match; this set might contain English sentences, or e-mail addresses, or TeX commands, or anything you like. You can then ask questions such as "Does this string match the pattern?", or "Is there a match for the pattern anywhere in this string?". You can also use REs to modify a string or to split it apart in various ways.

Läs mer: https://docs.python.org/3/howto/regex.html

Example of Regular Expression

Start of string

Literal dot

end of string

^ [a-zA-Z0-9_-]+ \. (jpg|png|gif) $

filename (letters, numbers, underscore, hyphens)

allowed extensions

Special Sequences (Explained)

`\d`

Matches any decimal digit; this is equivalent to the class `[0-9]`.

`\D`

Matches any non-digit character; this is equivalent to the class `[^0-9]`.

`\s`

Matches any whitespace character; this is equivalent to the class `[ \t\n\r\f\v]`.

`\S`

Matches any non-whitespace character; this is equivalent to the class `[^ \t\n\r\f\v]`.

`\w`

Matches any alphanumeric character; this is equivalent to the class `[a-zA-Z0-9_]`.

`\W`

Matches any non-alphanumeric character; this is equivalent to the class `[^a-zA-Z0-9_]`.

The Backslash Plague

As stated earlier, regular expressions use the backslash character (`'\'`) to indicate special forms or to allow special characters to be used without invoking their special meaning. This conflicts with Python's usage of the same character for the same purpose in string literals.

Let's say you want to write a RE that matches the string `\section`, which might be found in a LaTeX file. To figure out what to write in the program code, start with the desired string to be matched. Next, you must escape any backslashes and other metacharacters by preceding them with a backslash, resulting in the string `\\section`. The resulting string that must be passed to `re.compile()` must be `\\section`. However, to express this as a Python string literal, both backslashes must be escaped *again*.

Characters	Stage
`\section`	Text string to be matched
`\\section`	Escaped backslash for `re.compile()`
`"\\\\section"`	Escaped backslashes for a string literal

The solution is to use Python's raw string notation for regular expressions; backslashes are not handled in any special way in a string literal prefixed with `'r'`, so `r"\n"` is a two-character string containing `'\'` and `'n'`, while `"\n"` is a one-character string containing a newline. Regular expressions will often be written in Python code using this raw string notation.

In addition, special escape sequences that are valid in regular expressions, but not valid as Python string literals, now result in a `DeprecationWarning` and will eventually become a `SyntaxError`, which means the sequences will be invalid if raw string notation or escaping the backslashes isn't used.

Regular String	Raw string
`"ab*"`	`r"ab*"`
`"\\\\section"`	`r"\\section"`
`"\\w+\\s+\\1"`	`r"\w+\s+\1"`

Performing Matches

Once you have an object representing a compiled regular expression, what do you do with it? Pattern objects have several methods and attributes. Only the most significant ones will be covered here; consult the `re` docs for a complete listing.

Method/Attribute	Purpose
`match()`	Determine if the RE matches at the beginning of the string.
`search()`	Scan through a string, looking for any location where this RE matches.
`findall()`	Find all substrings where the RE matches, and returns them as a list.
`finditer()`	Find all substrings where the RE matches, and returns them as an iterator.

03

Regular Expression Code Examples

New Project
(Dependencies)

```
$ uv init
$ uv add pandas
```

Regex
Without Pandas

Regular Expression
(Import re)

```
import re
```

Import re == regex

Notera: Detta behövs INTE om du använder pandas.

Nästa sida visar hur du arbetar med regex utan Pandas

Regular Expression
(Import re)

```
import re

txt = "The rain in Spain 2"

#Check if the string contains any digits (numbers from 0-9):
x = re.findall("\d", txt)

print(x)

if x:
 print("Yes, there is at least one match!")
else:
 print("No match")
```

Source: https://www.w3schools.com/python/python_regex.asp

Regex
With Pandas

Regular Expression
(Preparing DataFrames)

```
import pandas as pd

if __name__ == '__main__':

    # Dataframe to be filtered
    df_text_and_number = pd.DataFrame({
        "text": ["apple123", "banana", "cherry456", "dragonfruit", "elderberry789"]
    })
```

Notera: import för ‘re’ behövs när du vill åkalla funktionerna för regex. Pandas förstår regex utan ‘compile’ funktionen då det är inbyggt.

Digits \d
With Pandas

Regular Expression
(Contain Digits 0-9)

```python
with_digits = df_text_and_number[
    df_text_and_number["text"].str.contains(r"\d", regex=True)
]

print(with_digits)
```

```
            text
0        apple123
2       cherry456
4  elderberry789
```

Regular Expression
(Contains ~NOT digits 0-9)

```
with_digits = df_text_and_number[
    ~df_text_and_number["text"].str.contains(r"\d", regex=True)
]

print(with_digits)
```

```
           text
1        banana
3  dragonfruit
```

Notera tecken: ~

Regular Expression
(Delete Digits with Replace())

```
df_text_and_number["filtered"] = df_text_and_number["text"].str.replace(r"\d", "", regex=True)
```

Notera: “” byter ut tal mot tom sträng

	text	filtered
0	apple123	apple
1	banana	banana
2	cherry456	cherry
3	dragonfruit	dragonfruit
4	elderberry789	elderberry

ABC123 → ABC

Regular Expression
(Create new table with digits)

```
df_text_and_number["digits"] = df_text_and_number["text"].str.extract(r"(\d+)")
```

Notera:

- extract() returnerar tal | NaN.
- extract() arbetar inom ‘groups’ med (\d)
- + tecknet är ‘recursive’ (prova ta bort den)

Om du inte tycker om detta, använd ‘findAll()’ istället

	text	filtered	digits
0	apple123	apple	123
1	banana	banana	NaN
2	cherry456	cherry	456
3	dragonfruit	dragonfruit	NaN
4	elderberry789	elderberry	789

Strings \s
With Pandas

Regular Expression
(Dataframe of Strings)

```
df_strings_whitespaces = pd.DataFrame({
    "text": [
        "hello world",
        "hello\tworld",
        "hello   world",
        "hello\nworld",
        "helloworld"
    ]
})
```

Notera: whitespace är inte bara " " utan kan också vara:
TAB = \t
Newline = \n

Regular Expression
(\s meaning)

\s

Betyder

[\t \n \r \f \v]

Tecken	Exempel	Vad som händer
space " "	"hello world"	vanligt mellanrum
\t (tab)	"hello\tworld"	hoppar ett tabsteg
\n (newline)	"hello\nworld"	ny rad
\r (carriage return)	"hello\rworld"	går till början av raden
\f (form feed)	"hello\fworld"	sidbrytning (används nästan aldrig idag)
\v (vertical tab)	"hello\vworld"	vertikal tab (väldigt ovanligt)

Regular Expression
(\s code)

```
df_strings_whitespaces["text"] = (df_strings_whitespaces["text"]
                                  .str.replace(r"\s+", " ", regex=True)
                                  .str.strip()
                                  )
```

Notera: strip tar bort extra mellanslag före & efter

Symbols & UTF-8
With Pandas

Regular Expression
(utf dataframe)

```
df_utf_problem = pd.DataFrame({
    "text": [
        "café",
        "naïve",
        "Malmö",
        "Göteborg",
        "Stockholm"
        "Stockholm123",
        "a"
        ""
    ]
})
```

Regular Expression
(\w meaning)

Vad `\w` inkluderar

Typ	Exempel
bokstäver	a b c ö ä å é ü
siffror	0–9
underscore	_

Regular Expression
(\w code)

```
result = df_utf_problem[df_utf_problem["text"].str.contains(r"\w", case=False, regex=True)]
```

```
0       café
1      naïve
2      Malmö
3   Göteborg
4  Stockholm
5          a
```

Notera att stockholm123 också kommer dyka upp här
(bilden är bara för liten)

Regular Expression
(\w code PROBLEM)

```
result = df_utf_problem[df_utf_problem["text"].str.contains(r"\w", case=False, regex=True)]
```

0	café
1	naïve
2	Malmö
3	Göteborg
4	Stockholma123

Notera att \w också accepterar - samt 0-9

Detta blir inte alltid önskvärt..

Regular Expression
(only chars + utf-8)

```
result2 = df_utf_problem[df_utf_problem["text"].str.strip().str.isalpha()]
```

	text
0	café
1	naïve
2	Malmö
3	Göteborg
4	Stockholm

LÖST!

a-z A-Z
With Pandas

Regular Expression
(Om du inte kräver UTF-8)

```
df_az_only = pd.DataFrame({
    "text": [
        "Stockholm",
        "Malmö",
        "Göteborg",
        "Åsa",
        "Äke",
        "Björn",
        "José",
        "Zoë",
        "Søren",
        "François"
    ]
})
```

Notera att vi här vill ta bort ‘åäö’ och bara visa resultat mellan a-z

Regular Expression
([a-z, A-Z])

```
var_az = df_az_only[df_az_only["text"].str.contains(r"^[a-z, A-Z]+$", case=False, regex=True)]
```

	text
0	Stockholm

`[a-z, A-Z]` = hämtar första matchning
`^` = från start till `$` *(hela strängen)*
`+` = en eller flera

THANKS !

Do you have any questions?
kristoffer.johansson@sti.se

CREDITS: This presentation template was created by Slidesgo, incluiding icons by Flaticon, and infographics & images by Freepik.